

LAB- 3

Name: Pratij Mishra

Batch: 21

Sap: 590028984

Q1: Kruskal Algorithm

Code:

```
#include <stdio.h>

struct Edge
{
    int u, v, weight;
};

// Find the parent of a vertex
int find(int parent[], int x)
{
    if (parent[x] == x)
        return x;

    return parent[x] = find(parent, parent[x]);
}

// Join two sets
void unionSet(int parent[], int rank[], int a, int b)
{
    a = find(parent, a);
    b = find(parent, b);

    if (a != b)
    {
        if (rank[a] < rank[b])
            parent[a] = b;
        else if (rank[a] > rank[b])
            parent[b] = a;
        else
        {
            parent[b] = a;
            rank[a]++;
        }
    }
}

// Sort edges according to weight
void sortEdges(struct Edge edges[], int E)
{

```

```

    for (int i = 0; i < E - 1; i++)
    {
        for (int j = 0; j < E - i - 1; j++)
        {
            if (edges[j].weight > edges[j + 1].weight)
            {
                struct Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

```

```

void kruskal(struct Edge edges[], int V, int E)
{
    int parent[V];
    int rank[V];

    // Initially, every vertex is its own set
    for (int i = 0; i < V; i++)
    {
        parent[i] = i;
        rank[i] = 0;
    }

    // Sort edges by weight
    sortEdges(edges, E);

    int edgeCount = 0;
    int totalWeight = 0;

    printf("\nEdges in Minimum Spanning Tree:\n");

    for (int i = 0; i < E && edgeCount < V - 1; i++)
    {
        int u = edges[i].u;
        int v = edges[i].v;

        // Check whether adding edge creates a cycle
        if (find(parent, u) != find(parent, v))
        {
            printf("%d -- %d Weight = %d\n",
                u, v, edges[i].weight);

            totalWeight += edges[i].weight;

            unionSet(parent, rank, u, v);
        }
    }
}

```

```

        edgeCount++;
    }
}

printf("Minimum Cost = %d\n", totalWeight);
}

int main()
{
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    struct Edge edges[E];

    printf("\nEnter edges (u v weight):\n");

    for (int i = 0; i < E; i++)
    {
        scanf("%d %d %d",
            &edges[i].u,
            &edges[i].v,
            &edges[i].weight);
    }

    kruskal(edges, V, E);

    return 0;
}

```

Output:

```
Enter number of vertices: 3
Enter number of edges: 3

Enter edges (u v weight):
0 1 1
0 2 3
1 2 2

Edges in Minimum Spanning Tree:
0 -- 1 Weight = 1
1 -- 2 Weight = 2
Minimum Cost = 3
PS C:\Users\USER\Desktop\programming in c\Lab-3> █
```